



Government  
Digital Service



Department for  
Science, Innovation  
& Technology

# **Aligning a Secure Software Development Pipeline (SSDP) with Secure by Design**

January 2026

# Contents

|                                                |    |
|------------------------------------------------|----|
| Background                                     | 3  |
| Purpose                                        | 3  |
| Scope                                          | 3  |
| 1. High-level pipeline requirements            | 5  |
| 2. Functional requirements by life cycle phase | 5  |
| 3. Non-functional requirements                 | 6  |
| 4. Planning and design                         | 7  |
| 5. Source code management                      | 8  |
| 6. Build phase                                 | 8  |
| 7. Test phase                                  | 9  |
| 8. Infrastructure and configuration            | 9  |
| 9. Deployment phase                            | 10 |
| 10. Operations and monitoring                  | 11 |
| 11. Governance                                 | 11 |
| 12. Metrics and reporting                      | 11 |
| More information                               | 11 |

# Background

A Secure Software Development Pipeline (SSDP) is a process that integrates security into every stage of software development. By adding security checks and controls early and throughout the pipeline, it helps identify and fix issues before software is released. This approach improves software security and quality while enabling teams to deliver applications and services efficiently.

[Secure by Design](#) is a government approach which embeds cyber security practices into digital delivery from the outset and consistently throughout the service life cycle. Its goal is to make security an integral, proactive part of all government digital systems, rather than an afterthought.

## Purpose

The purpose of this document is to align SSDP with the Secure by Design approach. It aims to help government organisations establish a consistent and reliable approach by integrating security into software delivery pipelines from the earliest stages of conception through to live operation and retirement.

This reduces reliance on individual judgment and makes security outcomes consistent across teams from the earliest stages of system development.

Architectural choices, development practices and operational controls should intentionally reduce risk based on service context and impact, rather than relying on late-stage testing or assurance.

## Scope

This guidance document can be used across all software systems and services developed, operated or commissioned by the government organisation using it, regardless of the delivery model or hosting environment.

Because insecure delivery mechanisms can undermine otherwise secure applications, the scope explicitly includes:

- Continuous Integration and Continuous Delivery or Deployment (CI/CD) pipelines
- cloud platforms
- container environments
- Infrastructure as Code (IaC)

Supply chain elements, including third-party libraries and supplier-developed components, are fully in scope as well. Government organisations must understand and manage the risks introduced by external technology products.

## Secure by Design on the UK Government Security website

- [Secure by Design principles](#)
- [Secure by Design policy](#)
- [Implementing Secure by Design](#)
- [Guide to adopting Secure by Design](#)

# 1. High-level pipeline requirements

An SSDP can be an effective security control in its own right, governing how changes are introduced into live environments. You should perform all deployments to production and other protected environments exclusively through the pipeline, and control manual changes through formally approved emergency processes.

The pipeline must:

- enforce separation of duties
- make sure there's appropriate authorisation at each stage
- prevent the promotion of unapproved or unverified changes

You should integrate security checks into pipeline stages, so that unsafe changes are prevented automatically rather than detected later. This should include:

- mandatory automated controls such as source code analysis, dependency and supply-chain scanning
- configuration and policy validation
- security testing appropriate to the system's risk profile

The pipeline must fail securely by default, blocking progression when defined security thresholds are not met. It must also make sure that security controls cannot be bypassed or disabled without explicit, auditable approval.

The pipeline must generate durable, auditable evidence of security activity, supporting ongoing assurance and assessment. This evidence must include:

- records of security checks performed
- their outcomes
- approvals granted
- changes deployed
- the identities of systems or individuals authorising changes

Audit data must be:

- protected against tampering
- retained in accordance with organisational requirements
- made readily available to support internal review, external audit and incident investigation

Because threats and technologies evolve, the pipeline should be adaptable without requiring a fundamental redesign. By making security controls, tooling and policies modular and configurable, you can:

- introduce new checks
- strengthen existing controls

- retire obsolete mechanisms with minimal disruption

By learning from emerging threats, threat intelligence, security incidents and assurance activities, you can continuously improve a pipeline.

This approach supports the following Secure by Design principles:

- 6. Design flexible architectures
- 8. Defend in depth
- 9. Embed continuous assurance
- 10. Make changes securely

## 2. Functional requirements by life cycle phase

Security requirements apply across the entire service life cycle and must operate as a coordinated system rather than isolated checkpoints. Security activities must be defined, owned and integrated for each life cycle phase, including:

- discovery
- design
- build
- test
- deployment
- operation
- change
- decommissioning

You should link controls, metrics and assurance outputs from each phase so that you can assess security posture continuously, not just at release or audit points.

You should focus on preventing structural weaknesses during the early life cycle stages through informed design decisions. This requires early threat modelling, security architecture reviews and explicitly accepting risks for design trade-offs.

Follow this with runtime monitoring, logging, alerting and incident response capabilities that are proportionate to the service's threat profile. You should document design assumptions and accepted risks and revisit them as part of an operational review.

In the later stages, concentrate on detection, response and recovery.

### Using automation and human judgement

You should use automation so that you're applying and reapplying consistent security controls wherever it's feasible to do so. That said, it's important to recognise that human judgement is essential for interpreting risk, approving exceptions and responding to incidents.

Use automated controls to enforce baseline security requirements and highlight deviations, and defined governance processes for human review, escalation, and decision-making. Exceptions to security requirements must be time-limited, risk-assessed and formally approved, with compensating controls applied where necessary.

## **Learning and improvement**

The integration of security requirements into your life cycle must include ways for lessons learned to inform your organisation's future design and delivery decisions. When you discover security incidents, vulnerability trends or assurance findings, you should systematically capture, analyse and feed them back into design standards, threat models and development practices.

The life cycle must include formal mechanisms for learning and improvement, so that any changes to the service make it more secure rather than bringing back previously-addressed risks.

This approach supports the following Secure by Design principles:

- 3. Adopt a risk-driven approach
- 5. Build in detect and respond security
- 9. Embed continuous assurance
- 10. Make changes securely

## **3. Non-functional requirements**

Non-functional requirements should keep security controls effective at scale, without impacting delivery or growth.

Usability requirements should ensure that findings and remediation guidance are easy to understand so delivery teams can act on them.

Reliability requirements should ensure that security tools do not rely on a single system.

This approach supports the following Secure by Design principles:

- 4. Design usable security controls
- 6. Design flexible architectures
- 8. Defend in depth

## 4. Planning and design

### Roles and responsibilities

Security should be a primary consideration in the planning and design phases, with named owners accountable for any security risks associated with the service. This ownership should be established before implementation begins and maintained throughout the service life cycle.

Security owners should have the authority to influence design decisions, accept or reject risk, and require remediation where controls are insufficient. It's important to clearly define roles and responsibilities for security, architecture and operations, and document them so there are no gaps in accountability.

### Threat modelling

Threat modelling must be performed for all new services and for any significant architectural, functional or environmental changes. This means design decisions will be informed by an understanding of likely threats, potential impacts and exposure.

Threat models must consider negative scenarios, supply-chain risks, insider threats and dependencies on external services. They should be updated as the service evolves and treated as living artefacts, with identified risks tracked through to mitigation, acceptance or retirement.

### Security architecture

Architectural decisions must favour simplicity, modularity and resilience. This allows security controls to evolve without needing to be fundamentally redesigned. Designs must support:

- clear trust boundaries
- well-defined interfaces
- isolation between components of differing sensitivity

Where possible, designs should reduce complexity and hidden dependencies that increase systemic risk. They should avoid tightly-coupled components that hinder effective security changes or recovery.

### Minimising the attack surface

Attack surface minimisation must be an explicit design objective. The initial design should only include required functionality, interfaces, data flows and privileges. Default configurations must be secure, with non-essential services disabled and access restricted by least privilege.



You should avoid unused, experimental or speculative features, or at least make them explicitly gated to reduce exposure. You should justify any expansion of the attack surface through documented risk assessment.

## **Security requirements**

You should explicitly capture security requirements as part of design documentation and acceptance criteria, and validate them before implementation proceeds.

Where feasible, you should document and test assumptions made during the design stage, particularly those relating to trust, identity, data protection and external dependencies.

Design reviews must include security expertise and must confirm that identified risks are either mitigated or consciously accepted by the accountable owner.

This approach supports the following Secure by Design principles:

- 1. Create responsibility for cyber security risk
- 3. Adopt a risk-driven approach
- 6. Design flexible architectures
- 7. Minimise the attack surface

## **5. Source code management**

Source code repositories must be treated as critical assets, with access restricted to authorised individuals and systems. You should review permissions regularly to make sure they follow least-privilege principles.

Code review processes must include security considerations and be supported by automated checks where possible. This helps identify insecure patterns early and consistently.

You should manage all dependencies, including open-source components, through approved mechanisms that make their origin, licensing and known vulnerabilities clear. You should prevent uncontrolled or one-off use of dependencies.

Never store secrets in source control. Use automated scanning and preventative controls to detect and block any accidental exposure.

This approach supports the following Secure by Design principles:

- 2. Source secure technology products
- 7. Minimise the attack surface
- 8. Defend in depth

## 6. Build phase

### Automation and isolation

You should fully automate build processes and make them repeatable, as well as executing them in isolated environments. This helps reduce human error and prevents them being modified without authorisation.

Using automation and isolation means builds always run the same way and are protected from interference. This supports a defence-in-depth approach, a key Secure by Design principle. You should make sure that any changes to the build configuration or inputs are controlled, auditable and subject to the same security standards as application code.

### Integrating security controls

You should tightly integrate security controls such as static application security testing (SAST), dependency and vulnerability scanning, and policy enforcement into the build stage. This helps identify weaknesses before artefacts are produced.

These controls should fail builds when defined risk thresholds are exceeded. To speed up investigation and remediation, all findings must be traceable back to specific builds, commits and dependency versions.

### Versioning and auditability

To protect the integrity of build artefacts and make their origins clear, they must be:

- uniquely versioned
- cryptographically signed
- stored in secure, access-controlled repositories

This allows downstream pipeline stages and runtime environments to verify that only trusted, approved artefacts are promoted and deployed.

The build stage must also generate comprehensive, auditable evidence of the checks performed and decisions taken. This should include:

- security scan results
- policy evaluations
- approval outcomes

This evidence supports end-to-end assurance across the delivery pipeline. It means compliance can be validated, analysis can be forensic and security controls can be continuously improved.

This approach supports the following Secure by Design principles:

- 7. Minimise the attack surface
- 8. Defend in depth
- 9. Embed continuous assurance

## 7. Test phase

Security testing is not just about confirming there are no known vulnerabilities. It should explicitly validate how effective detection and response mechanisms are, which might include logging, alerting and automated remediation controls. This includes verifying that security events are correctly generated, captured and acted upon.

### Using realistic conditions

Dynamic application security testing (DAST), API security testing, and misuse and abuse case testing must be embedded into test environments that mirror production as closely as possible in terms of configuration, data flows and security controls.

Testing in production-like environments increases confidence that any issues you identify are relevant and that deployed controls will behave as expected in real-world scenarios.

### Presenting test findings

Test results must be presented in a clear, prioritised and actionable way so that delivery teams can quickly understand risk, determine ownership and take corrective action.

Findings should be explained with guidance that covers severity, exploitability and remediation. Excessive false positives, noisy reporting and ambiguous results can make people less likely to trust security tooling.

### Keeping on top of defects

Any security defects you identify must be tracked, prioritised and resolved through standard delivery workflows alongside functional defects. This makes sure there's visibility and accountability. This approach makes security outcomes a shared responsibility and promotes continuous improvement.

This approach supports the following Secure by Design principles:

- 4. Design usable security controls
- 5. Build in detect and respond security
- 9. Embed continuous assurance

## 8. Infrastructure and configuration

### Using IaC

You should define, provision and manage infrastructure using IaC. This keeps infrastructure consistent, repeatable and auditable across environments. By using code-driven definitions, you can:

- reduce configuration drift
- reliably recreate environments
- provide a clear, reviewable record of the intended state of your system

### Infrastructure configurations

To minimise unnecessary exposure and limit the impact of a compromise, infrastructure configurations must enforce:

- least-privilege access
- strong identity boundaries
- network segmentation
- secure-by-default settings

When security controls are applied by default they can only be weakened through deliberate, justified changes.

### Using control layers

You should implement multiple, complementary layers of control to provide defence in depth. This should include:

- platform-level security features
- network controls
- host and workload protections
- monitoring capabilities

No single control should be relied upon in isolation. Instead, failures at one layer should be mitigated by safeguards at others.

### Controlling changes

All changes to infrastructure configuration should be version-controlled and subjected to the same review, testing and assurance processes as application code, including automated validation and security checks.

This means any changes are traceable and promotes accountability. It should also provide ongoing confidence that deployed infrastructure continues to meet security and compliance requirements.

This approach supports the following Secure by Design principles:

- 6. Design flexible architectures
- 7. Minimise the attack surface
- 8. Defend in depth
- 9. Embed continuous assurance

## 9. Deployment phase

### Deployment security controls

Deployment mechanisms must prevent unauthorised and unverified artefacts from entering live environments. Their controls should validate the integrity and provenance of artefacts, as well as their compliance with defined security requirements before release. There's more about this in AWS's Well-Architected guidance, [Deploy exclusively from trusted artifact repositories](#). Google Cloud's Supply-chain security and Binary Authorization guidance, [Using Binary Authorization to boost supply chain security](#) and Microsoft Azure's guidance on Artifact Signing and secure DevOps with Azure Policy and Microsoft Defender for Cloud recommendations, [Azure Artifact Signing](#).

To reduce the likelihood of unsafe releases taking place, security controls such as approval gates, automated policy enforcement and environment validation checks should be layered throughout the deployment process. Use automated controls to enforce baseline requirements consistently. Manual approvals should be reserved for high-risk changes where it's best to use human judgement.

### Deployment strategies

You should design deployment strategies to support rapid rollback, failover and recovery in the event of defective releases. Techniques such as blue-green deployments, canary releases and feature toggles can be helpful to limit blast radius and speed up fixes without disrupting the service for longer than is necessary.

### Audit trails

All deployment activity should generate clear, tamper-resistant audit trails. These should link deployed changes to whoever approved them, source artefacts, configuration versions and associated risk decisions.

Effective audit trails:

- provide accountability
- support incident response and compliance needs
- enable continuous improvement of deployment controls

This approach supports the following Secure by Design principles:

- 6. Design flexible architectures
- 8. Defend in depth
- 9. Embed continuous assurance
- 10. Make changes securely

## 10. Operations and monitoring

Once a service is live, it should be continuously monitored for indicators of compromise, misconfiguration and emerging vulnerabilities.

Logging and monitoring should be effective enough to help with investigation, containment and recovery, and must be protected from tampering.

Vulnerability management processes should make sure that newly-identified issues are assessed for impact and addressed through controlled change.

You should feed back any operational insights into planning, design and pipeline controls. This will help embed continuous improvement.

This approach supports the following Secure by Design principles:

- 5. Build in detect and respond security
- 8. Defend in depth
- 9. Embed continuous assurance
- 10. Make changes securely

## 11. Governance

Embedding governance into pipelines:

- reduces reliance on periodic reviews
- strengthens accountability across delivery teams

Well-structured governance makes sure that security risk is owned and managed at the appropriate organisational level.

Policies should be centrally defined but consistently applied, with deviations explicitly documented and time limited.

This approach supports the following Secure by Design principles:

- 1. Create responsibility for cyber security risk
- 3. Adopt a risk-driven approach

## 12. Metrics and reporting

Metrics help show how strong an organisation's security posture is and how well its controls work. This helps people make informed decisions.

Operational metrics help teams detect issues and fix them. Reporting should surface significant risks promptly and avoid overwhelming teams with unhelpful data.

Metrics should evolve over time to reflect changes in both the threat landscape and delivery practices.

This approach supports the following Secure by Design principles:

- 3. Adopt a risk-driven approach
- 4. Design usable security controls
- 9. Embed continuous assurance

## More information

- [Secure by Design principles \(UK Government Security\)](#)
- [Secure by Design activities \(UK Government Security\)](#)
- [Secure the build and deployment pipeline \(NCSC\)](#)
- [Threat modelling \(NCSC\)](#)
- [Secure development principles \(NCSC\)](#)